GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning

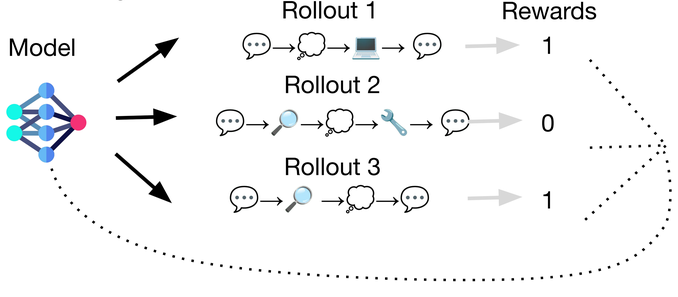
Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, Omar Khattab

Motivation

As FLOPS get cheaper, progress in many AI capabilities will be bottlenecked by sample efficiency!

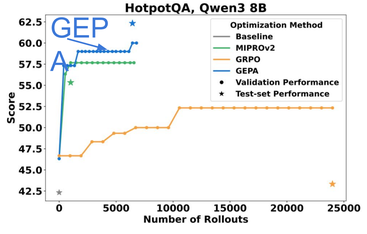
Low availability of domain specific knowledge resources

Expensive rollouts: either the LLM workflow for the task or the task metric is slow/expensive to execute

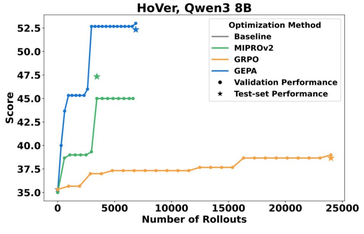


There was a lot of info in each rollout, but we only took a 0/1 score propagated via gradient descent. Can we use the other info?

GEPA is Highly Sample Efficient!



(a) HotpotQA, Qwen3 8B



(b) HoVer, Qwen3 8B

GEPA Generated Prompts are Incredibly Detailed!

Given the fields `question`, `summary_1`, produce the fields `query`.

You will be given two input fields: `question` and `summary_1`. Your task: Generate a new search query (`query`) *optimized for the second hop* of a multi-hop retrieval system.

- The original user question is typically complex and requires information from multiple documents to answer.
- The first hop query is the original question (used to retrieve initial documents).
- Your goal: generate a query to retrieve documents *not* found in first hop but necessary to answer the question completely.

Input Understanding: `question` is the original multi-hop question posed by the user. `summary_1` is a concise summary of information from a document retrieved in the first hop, which partially addresses the question.

Purpose and Context:

- Your generated `query` aims to find the *missing pieces* of information needed to fully answer the question. . . .
- The query must retrieve relevant documents *NOT* found in first hop . . . for final answer extraction.

Key Observations and Lessons:

- First-hop documents often cover one entity or aspect.
- Remaining relevant documents often involve connected or higher-level concepts mentioned in `summary_1` but not explicitly asked in the original question. The `query` should target these *missing*, but logically linked, documents.
- Avoid merely paraphrasing the original question or restating known facts from `summary_1`.
- Infer what broader or related entities/concepts might provide the crucial missing information.
- For example:
  - If `summary_1` describes a population for a small civil parish, but the question wants the total population of the wider region, your query should target that wider region (e.g., "Madeira archipelago population in 2011").
  - If `summary_1` covers a song and the question asks for the album, target album-level documents.

How to Build the Query:

- Identify entities or topics mentioned in `summary_1` that are related but different from first-hop documents.
- Reframe the query to explicitly mention these broader or related entities *connected to the original question*.
- Include relevant key context from the question to maintain specificity, but shift focus to the missing piece.
- The goal is to retrieve documents that link or complement what was retrieved initially.

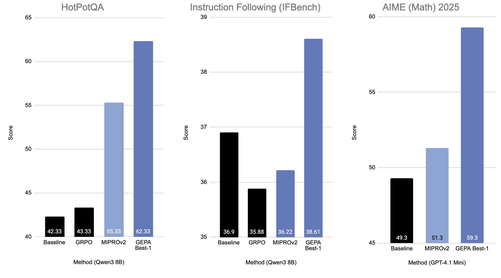
Practical Strategy:

- Read the `summary_1` carefully to spot references to bigger contexts or other entities not covered in the first hop.
- Ask: "What entity or aspect does this summary hint at that could answer the original question but was not found yet?"
- Formulate a precise, focused factual query targeting that entity or concept to retrieve the missing documents.

Output:

- Produce `query` as a clear, concise question or keyword phrase designed for efficient retrieval of second-hop documents.
- Ensure the query relates logically to the original question while targeting the broader or complementary knowledge identified in `summary_1`. . . . Do not include the original question or simply rephrase it. Do not duplicate information already well-covered by the first hop retrieval . . .

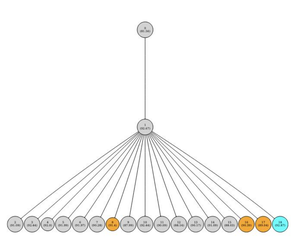
GEPA is Highly Sample Efficient!



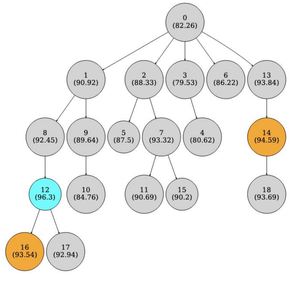
Qwen3 8B	HotpotQA	IFBench	Hover	PUPA	AIME-2025	LiveBench-Math	Aggregate	Improvement
Baseline	42.33	36.90	35.33	80.82	27.33	48.70	45.23	—
GRPO	43.33	35.88	38.67	86.66	**38.00**	51.26	48.91	+3.68
MIPROv2	55.33	36.22	47.33	81.55	20.00	46.60	47.84	+2.61
GEPA	62.33	**38.61**	**52.33**	**91.85**	32.00	**51.95**	**54.85**	**+9.62**
GEPA+Merge	**64.33**	28.23	51.67	86.26	32.00	**51.95**	52.40	+7.17
Total optimization budget (# rollouts)								
GEPA (+Merge)	6871	3593	7051	2426	1839	1839	3936	—
GRPO	24000	24000	24000	24000	24000	24000	24000	—

GPT-4.1 Mini	HotpotQA	IFBench	Hover	PUPA	AIME-2025	LiveBench-Math	Aggregate	Improvement
Baseline	38.00	47.79	46.33	78.57	49.33	58.20	53.03	—
Trace (OptoPrime)	60.33	51.19	46.00	74.18	45.33	60.74	56.30	+3.27
MIPROv2-No-Demos	38.00	52.04	51.33	91.85	48.67	60.97	57.14	+4.11
MIPROv2	58.00	49.15	48.33	83.37	51.33	61.84	58.67	+5.64
TextGrad	62.33	48.64	47.67	85.68	46.67	63.84	59.14	+6.11
GEPA	**69.00**	52.72	51.67	94.47	**59.33**	**64.13**	65.22	+12.19
GEPA+Merge	65.67	**55.95**	**56.67**	**96.46**	**59.33**	**64.13**	**66.36**	**+13.33**
Optimized with Qwen3-8B, evaluated on GPT-4.1-Mini								
GEPA-Qwen-Opt	65.67	49.83	54.67	90.05	52.67	59.31	62.03	+9.00

Pareto-based Candidate Selection is Important



(a) SelectBestCandidate Strategy



(b) Pareto-based candidate sampling.

Qwen3 8B	HotpotQA	IFBench	Hover	PUPA	Aggregate	Improvement
Baseline	42.33	36.90	35.33	80.82	48.84	—
SelectBestCandidate	58.33	30.44	45.33	85.45	54.89	+6.05
BeamSearch	57.33	36.39	41.00	81.08	53.95	+5.11
GEPA	**62.33**	**38.61**	**52.33**	**91.85**	**61.28**	**+12.44**

GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning

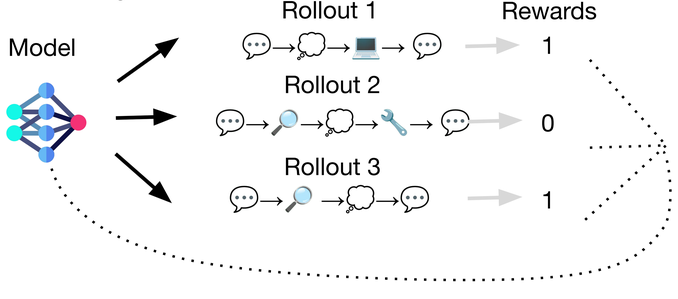
Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, Omar Khattab

Motivation

1. As FLOPS get cheaper, progress in many AI capabilities will be bottlenecked by sample efficiency!

Low availability of domain specific knowledge resources

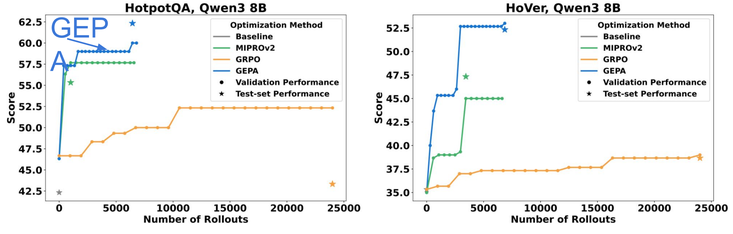
Expensive rollouts: either the LLM workflow for the task or the task metric is slow/expensive to execute



There was a lot of info in each rollout, but we only took a 0/1 score propagated via gradient descent. Can we use the other info?

2. How do we "train" frontier closed-source models?

GEPA is Highly Sample Efficient!



(a) HotpotQA, Qwen3 8B (b) HoVer, Qwen3 8B

GEPA Generated Prompts are Incredibly Detailed!

Given the fields `question`, `summary_1`, produce the fields `query`.

You will be given two input fields: `question` and `summary_1`. Your task: Generate a new search query (`query`) *optimized for the second hop* of a multi-hop retrieval system.

- The original user question is typically complex and requires information from multiple documents to answer.
- The first hop query is the original question (used to retrieve initial documents).
- Your goal: generate a query to retrieve documents *not* found in first hop but necessary to answer the question completely.

Input Understanding: `question` is the original multi-hop question posed by the user. `summary_1` is a concise summary of information from a document retrieved in the first hop, which partially addresses the question.

Purpose and Context:

- Your generated `query` aims to find the *missing pieces* of information needed to fully answer the question. . . .
- The query must retrieve relevant documents *NOT* found in first hop . . . for final answer extraction.

Key Observations and Lessons:

- First-hop documents often cover one entity or aspect.
- Remaining relevant documents often involve connected or higher-level concepts mentioned in `summary_1` but not explicitly asked in the original question. The `query` should target these *missing*, but logically linked, documents.
- Avoid merely paraphrasing the original question or restating known facts from `summary_1`.
- Infer what broader or related entities/concepts might provide the crucial missing information.
- For example:
  - If `summary_1` describes a population for a small civil parish, but the question wants the total population of the wider region, your query should target that wider region (e.g., "Madeira archipelago population in 2011").
  - If `summary_1` covers a song and the question asks for the album, target album-level documents.

How to Build the Query:

- Identify entities or topics mentioned in `summary_1` that are related but different from first-hop documents.
- Reframe the query to explicitly mention these broader or related entities *connected to the original question*.
- Include relevant key context from the question to maintain specificity, but shift focus to the missing piece.
- The goal is to retrieve documents that link or complement what was retrieved initially.

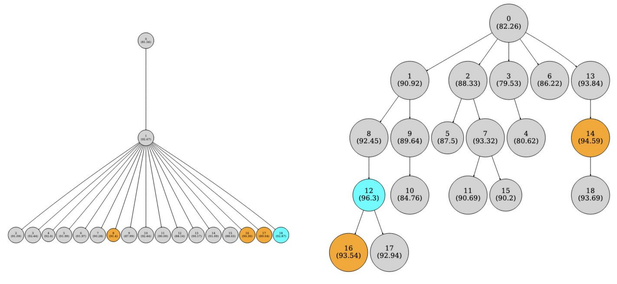
Practical Strategy:

- Read the `summary_1` carefully to spot references to bigger contexts or other entities not covered in the first hop.
- Ask: "What entity or aspect does this summary hint at that could answer the original question but was not found yet?"
- Formulate a precise, focused factual query targeting that entity or concept to retrieve the missing documents.

Output:

- Produce `query` as a clear, concise question or keyword phrase designed for efficient retrieval of second-hop documents.
- Ensure the query relates logically to the original question while targeting the broader or complementary knowledge identified in `summary_1`. . . . Do not include the original question or simply rephrase it. Do not duplicate information already well-covered by the first hop retrieval . . .

Pareto-based Candidate Selection is Important

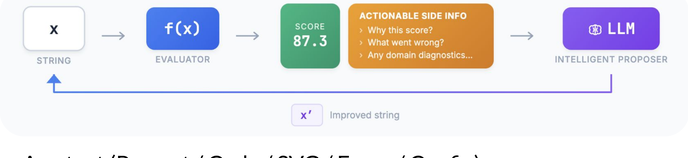


(a) SelectBestCandidate Strategy (b) Pareto-based candidate sampling.

Qwen3 8B	HotpotQA	IFBench	Hover	PUPA	Aggregate	Improvement
Baseline	42.33	36.90	35.33	80.82	48.84	—
SelectBestCandidate	58.33	30.44	45.33	85.45	54.89	+6.05
BeamSearch	57.33	36.39	41.00	81.08	53.95	+5.11
GEPA	**62.33**	**38.61**	**52.33**	**91.85**	**61.28**	**+12.44**

Prompt optimization to text optimization

`optimize_anything`: A Universal API for Optimizing any Text Parameter



- Any text (Prompt / Code / SVG / Essay / Config)
- Any task (Single-task search, Multi-task search, Generalization)
- Any algorithm (ShinkaEvolve, OpenEvolve, Combee)

GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning

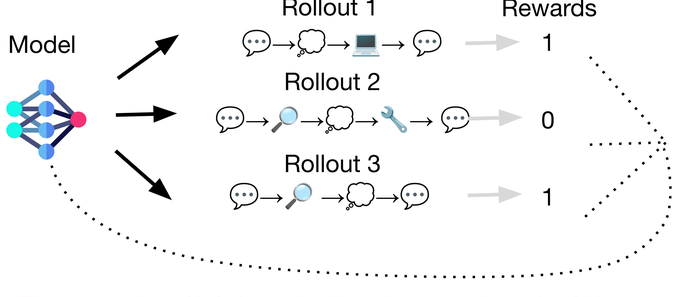
Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, Omar Khattab

Poor Sample Efficiency Blocks Agent Abilties

1. As FLOPS get cheaper, progress in many AI capabilities will be bottlenecked by sample efficiency!

Low availability of domain specific knowledge resources

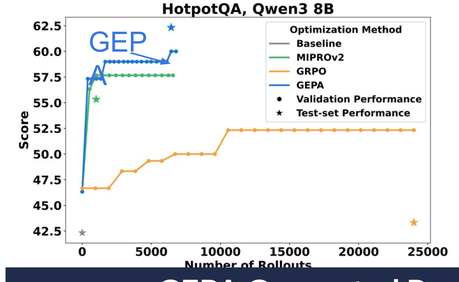
Expensive rollouts: either the LLM workflow for the task or the task metric is slow/expensive to execute

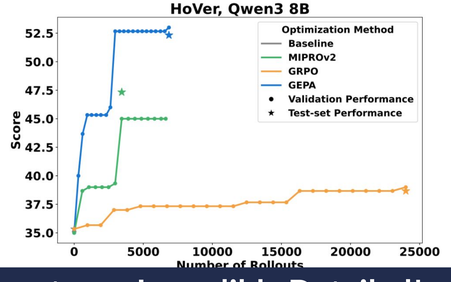


There was a lot of info in each rollout, but we only took a 0/1 score propagated via gradient descent. Can we use the other info?

2. How do we "train" frontier closed-source models?

GEPA is Highly Sample Efficient!





GEPA Generated Prompts are Incredibly Detailed!

Given the fields `question`, `summary_1`, produce the fields `query`.

You will be given two input fields: `question` and `summary_1`. Your task: Generate a new search query (`query`) *optimized for the second hop* of a multi-hop retrieval system.

- The original user question is typically complex and requires information from multiple documents to answer.
- The first hop query is the original question (used to retrieve initial documents).
- Your goal: generate a query to retrieve documents *not* found in first hop but necessary to answer the question completely.

Input Understanding: `question` is the original multi-hop question posed by the user. `summary_1` is a concise summary of information from a document retrieved in the first hop, which partially addresses the question.

Purpose and Context:

- Your generated `query` aims to find the *missing pieces* of information needed to fully answer the question. . . .
- The query must retrieve relevant documents *NOT* found in first hop . . . for final answer extraction.

Key Observations and Lessons:

- First-hop documents often cover one entity or aspect.
- Remaining relevant documents often involve connected or higher-level concepts mentioned in `summary_1` but not explicitly asked in the original question. The `query` should target these *missing*, but logically linked, documents.
- Avoid merely paraphrasing the original question or restating known facts from `summary_1`.
- Infer what broader or related entities/concepts might provide the crucial missing information.
- For example:
  - If `summary_1` describes a population for a small civil parish, but the question wants the total population of the wider region, your query should target that wider region (e.g., "Madeira archipelago population in 2011").
  - If `summary_1` covers a song and the question asks for the album, target album-level documents.

How to Build the Query:

- Identify entities or topics mentioned in `summary_1` that are related but different from first-hop documents.
- Reframe the query to explicitly mention these broader or related entities *connected to the original question*.
- Include relevant key context from the question to maintain specificity, but shift focus to the missing piece.
- The goal is to retrieve documents that link or complement what was retrieved initially.

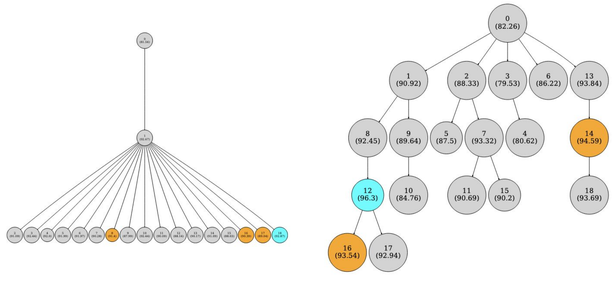
Practical Strategy:

- Read the `summary_1` carefully to spot references to bigger contexts or other entities not covered in the first hop.
- Ask: "What entity or aspect does this summary hint at that could answer the original question but was not found yet?"
- Formulate a precise, focused factual query targeting that entity or concept to retrieve the missing documents.

Output:

- Produce `query` as a clear, concise question or keyword phrase designed for efficient retrieval of second-hop documents.
- Ensure the query relates logically to the original question while targeting the broader or complementary knowledge identified in `summary_1`. . . . Do not include the original question or simply rephrase it. Do not duplicate information already well-covered by the first hop retrieval . . .

Pareto-based Candidate Selection is Important



(a) SelectBestCandidate Strategy (b) Pareto-based candidate sampling.

Qwen3 8B	HotpotQA	IFBench	Hover	PUPA	Aggregate	Improvement
Baseline	42.33	36.90	35.33	80.82	48.84	—
SelectBestCandidate	58.33	30.44	45.33	85.45	54.89	+6.05
BeamSearch	57.33	36.39	41.00	81.08	53.95	+5.11
GEPA	**62.33**	**38.61**	**52.33**	**91.85**	**61.28**	**+12.44**

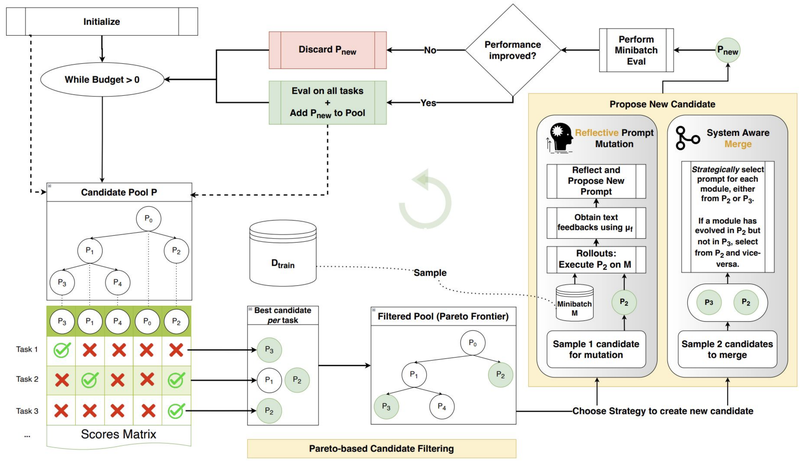
Prompt optimization to text optimization

`optimize_anything`: A Universal API for Optimizing any Text Parameter

Any text (Prompt / Code / SVG / Essay / Config)

↓

GEPA: Reflective Evolution and Pareto-based Candidate Selection



Examples

Reward Design with Language Models

Minae Kwon, Sang Michael Xie, Kalesha Bullard, Dorsa Sadigh

Goal

Make it *easier* to communicate our objectives to RL agents by using natural language

I want a versatile negotiator

Description

Alice : propose: book=1 hat=1 ball=0
Bob : propose: book=0 hat=1 ball=0
Alice : propose: book=1 hat=0 ball=1
Agreement!
Alice : 4 points
Bob : 5 points

Alice is versatile because she suggested different proposals

Handful of Examples w. Explanation

Prompt

Task Description

Alice and Bob are negotiating how to split a set of books, hats, and balls.

Example from User Describing Objective

Alice : propose: book=1 hat=1 ball=0
Bob : propose: book=0 hat=1 ball=0
Alice : propose: book=1 hat=0 ball=1

Is Alice a versatile negotiator?
Yes, because she suggested different proposals.

Episode Outcome

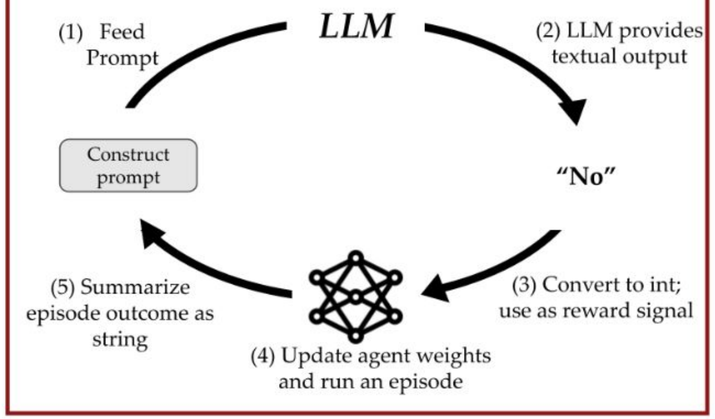
Alice : propose: book=1 hat=1 ball=0
Bob : propose: book=0 hat=1 ball=0
Alice : propose: book=1 hat=1 ball=0

Question

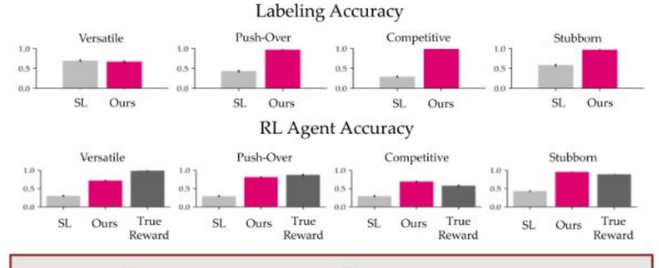
Is Alice a versatile negotiator?

Our Framework

Key Insight: Use LLM as a proxy reward



Automatic Evaluation

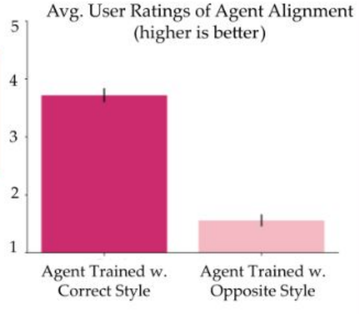


Takeaway: ✅Outperform SL by 46% ✅Underperform True Reward by 4%

Pilot User Evaluation

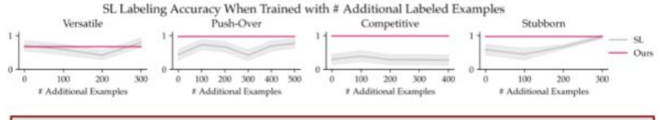
Takeaway:
✅ Humans find our agents to be more aligned than an agent trained with a different objective.

Examples of user chosen styles: ambitious, polite, considerate



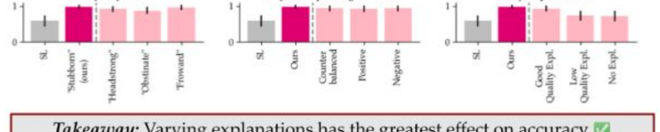
Further Analysis

How much data is required?



Takeaway: SL needs on the order of 100s of more examples to be accurate ✅

How much prompt design is required?



Takeaway: Varying explanations has the greatest effect on accuracy ✅